

ПРОЕКТНОЕ РЕШЕНИЕ

**Унифицированный бот для мониторинга образовательного
процесса**

Ruobr Unified Bot (Telegram + ВКонтакте)

Барданос В.В.

Версия 2.1 | Май 2026

Конфиденциально

1. Глоссарий, термины и сокращения

Термин	Определение
Ruobr	Региональная система управления образовательным процессом (ruobr.ru), Новосибирская область
TG / Telegram	Мессенджер Telegram (telegram.org)
VK / ВКонтакте	Социальная сеть ВКонтакте (vk.com)
aiogram 3.x	Асинхронный фреймворк для Telegram Bot API на Python
vkbottle 4.x	Асинхронный фреймворк для VK Bot API на Python
FSM	Finite State Machine — конечный автомат для многошаговых диалогов
LRU Cache	Кэш с вытеснением наименее используемых записей (Least Recently Used)
TTL	Time To Live — время жизни записи в кэше
Fernet	Симметричное шифрование AES-128-CBC (библиотека cryptography, Python)
ПФДО	Персональный фонд дополнительного образования
QRATOR	Защита от ботов на ruobr.ru, блокирует небраузерные User-Agent
SOCKS5	Протокол прокси-сервера для туннелирования TCP-соединений
WAL	Write-Ahead Logging — режим журналирования SQLite
Rate Limiting	Ограничение частоты запросов (sliding window algorithm)
Middleware	Промежуточный слой обработки запросов

Polling	Механизм получения обновлений (long polling для TG и VK)
---------	--

2. Лист версионности

Версия	Дата	Описание изменений	Автор
1.0	15.01.2025	Начальная версия: Telegram-бот с уведомлениями	Барданос В.В.
1.5	01.02.2025	Добавлен VK-бот, единая архитектура TG+VK	Барданос В.В.
1.8	01.03.2025	Дни рождения, настройка порогов баланса, дедупликация	Барданос В.В.
2.0	20.04.2026	Миграция на NAS, SOCKS5-прокси, обход QRATOR, объединение аккаунтов	Барданос В.В.

3. Общие положения

3.1. Цель проекта

Целью проекта является разработка и развёртывание единого бота для Telegram и ВКонтакте, обеспечивающего автоматический мониторинг образовательного процесса в системе «Руобр» (ruobr.ru)

Новосибирской области. Бот предоставляет родителям мгновенный доступ к информации о расписании, домашних заданиях, оценках, питании детей и балансе льготного питания. Ключевое преимущество — двухканальная доставка уведомлений: пользователь может получить уведомления одновременно в Telegram и ВКонтакте, с независимой настройкой типов уведомлений для каждого канала.

3.2. Основные параметры проекта

Параметр	Значение
Наименование	Ruobr Unified Bot
Назначение	Мониторинг образовательного процесса (расписание, ДЗ, оценки, питание, ДР)
Платформы	Telegram, ВКонтакте
Язык разработки	Python 3.12
Архитектура	Модульный монолит, backend-only (без веб-интерфейса)
База данных	SQLite (WAL),aiosqlite
Развертывание	Docker (single container), Synology NAS + VPS (SOCKS5-прокси)
Репозиторий	github.com/sakurka-cmd/ruobr-unified-bot
Организация	Барданос В.В.

3.3. Задачи проекта

1. Авторизация пользователей в системе «Руобр» через бота с безопасным хранением учётных данных (Fernet-шифрование).
2. Предоставление информации о расписании уроков, домашних заданиях, оценках за последние 14 дней, балансе питания и сертификатах ПФДО.
3. Фоновый мониторинг изменений с адаптивными интервалами проверки: 15 минут во время уроков, 60 минут — вне уроков.
4. Двухканальная доставка уведомлений (Telegram + ВКонтакте) с независимой настройкой по каналам.
5. Уведомления о низком балансе питания с настраиваемым порогом на каждого ребёнка.
6. Уведомления о фактах питания ребёнка (что ел, стоимость) с автоматическим определением из меню.
7. Уведомления о днях рождения одноклассников (режим «завтра» и «еженедельный дайджест»).
8. Обход защиты QRATOR на ruobr.ru через подмену User-Agent httpx.
9. Поддержка файловых вложений в домашних заданиях: скачивание и пересылка изображений (BufferedInputFile) и документов через API «Руобр».
10. Работа Telegram-бота через SOCKS5-прокси при блокировке TG провайдером.
11. Кросс-линковка аккаунтов TG и VK для единого профиля пользователя.
12. Автоматическая миграция схемы БД при обновлении (old schema → new schema с AUTOINCREMENT).

13. Дедупликация уведомлений через notification_history с уникальными ключами.

3.4. RICEF

Интерфейс (I)	Тип	Описание
/set_login	Команда	Авторизация в системе «Руобр» (ввод логина и пароля через FSM)
/balance	Команда	Баланс питания по детям
/schedule	Команда	Расписание на сегодня/завтра/неделю
/homework	Команда	Домашние задания на выбранную дату
/marks	Команда	Оценки за последние 14 дней
/achievements	Команда	Дополнительное образование (направления, сертификаты ПФДО)
/guide	Команда	Справочник школы (адрес, телефон, учителя)
/settings	Команда	Настройки уведомлений (TG)
/link_vk	Команда	Привязка VK-аккаунта к TG (кросс-линковка через код)
Inline-клавиатура	Callback	Переключение уведомлений (баланс, оценки, питание, ДР)
VK-клавиатура	Текст	Аналогичные функции через VK-кнопки
Кнопка «Баланс питания»	Текст	Отображение баланса питания (VK)

4. Характеристика объекта автоматизации

Объектом автоматизации является процесс информирования родителей (законных представителей) об образовательной активности их детей в системе «Руобр» (ruobr.ru). В настоящее время для получения информации родителю необходимо вручную заходить на портал через веб-браузер, авторизовываться и последовательно проверять различные разделы (расписание, оценки, питание). Это неудобно, особенно при наличии нескольких детей, и не обеспечивает оперативного реагирования на изменения.

Бот автоматизирует процесс сбора и доставки информации: он с заданной периодичностью обращается к API системы «Руобр», получает обновлённые данные и доставляет их пользователю через мессенджеры Telegram и ВКонтакте. Система поддерживает произвольное количество детей на аккаунте и независимую настройку каналов уведомлений для каждого пользователя.

4.1. Бизнес-процесс «To Be»

Основной бизнес-процесс работы системы включает следующие этапы:

№	Этап	Описание
1	Авторизация	Пользователь отправляет /set_login, вводит логин и пароль «Руобр» через FSM. Бот валидирует учётные данные через API и сохраняет пароль в зашифрованном виде (Fernet AES-128-CBC). Создаёт запись

		пользователя в БД с привязкой к chat_id (TG) и/или peer_id (VK).
2	Запрос информации	Пользователь запрашивает информацию через команды (/schedule, /homework, /marks, /balance, /achievements, /guide). Бот через RuobrClient (AsyncRuobr) авторизуется в «Руобр» и получает данные. Результат форматируется и отправляется пользователю.
3	Фоновый мониторинг	NotificationService группирует пользователей по учётным данным (один API-вызов на группу), с заданным интервалом проверяет балансы, оценки, питание и дни рождения. При обнаружении изменений — отправляет уведомления в настроенные каналы (TG/VK).
4	Настройка уведомлений	Пользователь через inline-клавиатуру (TG) или VK-клавиатуру настраивает 4 типа уведомлений для каждого канала независимо: баланс, оценки, питание, дни рождения.
5	Кросс-линковка	Пользователь может привязать TG-аккаунт к VK

		(или наоборот) через одноразовый 8-символьный код (действителен 10 минут). После привязки настройки и учётные данные объединяются в единый профиль.
6	Проксирование TG	При наличии переменной окружения BOT_PROXY бот создаёт AiohttpSession с SOCKS5-прокси (через aiohttp-socks ProxyConnector или нативный параметр проху в aiogram 3.15+), обеспечивая работу TG-бота в условиях блокировки Telegram провайдером.

4.2. Схема базы данных

База данных SQLite содержит 8 таблиц, обеспечивающих хранение пользователей, настроек уведомлений, истории, порогов и состояний FSM:

Таблица	Структура	Назначение
users	id (PK), chat_id (UNIQUE), peer_id (UNIQUE), login, password_encrypted, created_at, updated_at	Основная таблица пользователей. Один пользователь может иметь привязку к TG (chat_id) и VK (peer_id) одновременно.
notification_settings	user_id (FK), channel (tg/vk), balance_enabled, marks_enabled,	Настройки уведомлений по каналам. Две строки на пользователя (одна для TG,

	food_enabled, birthday_enabled, PK(user_id, channel)	одна для VK).
thresholds	user_id (FK), child_id, threshold (default 300), updated_at, PK(user_id, child_id)	Порог баланса питания для каждого ребёнка. При падении ниже порога — уведомление.
notification_history	id (PK), user_id, notification_type, notification_key, channel, created_at, UNIQUE(user_id, type, key, channel)	История отправленных уведомлений для дедупликации. Очищается автоматически (30 дней).
birthday_settings	user_id (FK), child_id, enabled, mode (tomorrow/weekly), notify_weekday, notify_hour, notify_minute, PK(user_id, child_id)	Настройки уведомлений о днях рождения одноклассников.
fsm_states	user_id (PK), state, data, updated_at	Состояния FSM для Telegram (авторизация, настройка порогов, дедлайн).
vk_fsm_states	peer_id (PK), state, data, updated_at	Состояния FSM для ВКонтакте.
link_codes	code (PK), user_id (FK), source (tg/vk), created_at	Одноразовые коды для кросс-линковки TG и VK аккаунтов. Действительны 10 минут.

5. Архитектурные решения

5.1. Функционально-компонентная архитектура

Система реализована по паттерну «модульный монолит» — единое приложение без микросервисной декомпозиции, с чётким разделением на модули. Отсутствие веб-фронта — интерфейсом являются мессенджеры Telegram и ВКонтакте. Архитектура обеспечивает полную независимость каналов: падение TG не влияет на VK и наоборот, так как каждый бот запускается как отдельная asyncio-задача.

5.2. Компоненты системы

Файл	Назначение
main.py	Точка входа. Инициализация BOT (aiogram), VK-бота (vkbottle), NotificationService. Управление прокси, shutdown, сигналы.
bot/config.py	Конфигурация из env-переменных. Валидация обязательных параметров (BOT_TOKEN или VK_TOKEN, ENCRYPTION_KEY).
bot/database.py	Пул соединений SQLite (aiosqlite, WAL). CRUD пользователей, настройка уведомлений, порогов, история, link_codes. Автоматическая миграция old → new schema, healing peer_id.
bot/encryption.py	Шифрование/дешифрование паролей (Fernet, AES-128-CBC). Глобальный сервис-синглтон.
bot/credentials.py	Единая точка расшифровки учётных данных (safe_decrypt). Пароль затирается

	из памяти после использования.
bot/middlewares.py	RateLimitMiddleware (sliding window, 30 req/min), AuthMiddleware (подгрузка UserConfig), LoggingMiddleware.
bot/handlers/auth.py	FSM-авторизация (/set_login), управление настройками уведомлений (inline-клавиатура), кросс-линковка (/link_vk), пороги баланса.
bot/handlers/schedule.py	Расписание уроков (/schedule), домашние задания (/homework), скачивание файлов ДЗ.
bot/handlers/balance.py	Баланс питания (/balance), выбор ребёнка, форматирование.
bot/handlers/birthday.py	Настройка уведомлений о днях рождения (FSM: выбор ребёнка, режим, день недели, время).
bot/vk/handlers.py	Все обработчики VK: расписание, ДЗ, оценки, баланс, настройки уведомлений, авторизация, дедупликация по message.id.
bot/vk/keyboards.py	VK-клавиатуры (основное меню, настройки, уведомления, выбор детей, дни рождения).
bot/services/notifications.py	NotificationService: фоновый мониторинг баланса, оценок, питания, дней рождения. Адаптивные интервалы, jitter, baseline, группировка по учётным данным.
bot/services/ruobr_client.py	RuobrClient: асинхронный клиент API «Руобр» (AsyncRuobr). Повторные попытки, таймауты, обработка ошибок. Скачивание файлов ДЗ.
bot/services/cache.py	MemoryCache: in-memory LRU-кэш с TTL (OrderedDict). 5 кэшей: children, timetable,

	food, threshold, birthday_settings.
bot/states.py	FSM-состояния: LoginStates, ThresholdStates, NotificationStates, BirthdaySettingsStates, LinkVKStates.
bot/utils/formatters.py	Утилиты форматирования: truncate_text, extract_dish_names, parse_complex_menu, normalize_date_to_iso.

5.3. Модель данных пользователя

Ключевая архитектурная особенность — единый пользовательский аккаунт с двумя каналами доставки. Dataclass `UserConfig` содержит 8 флагов уведомлений: 4 для TG (`enabled`, `marks_enabled`, `food_enabled`, `birthday_enabled`) и 4 для VK (`vk_balance_enabled`, `vk_marks_enabled`, `vk_food_enabled`, `vk_birthday_enabled`). В базе данных это хранится как две строки в таблице `notification_settings` с полями `channel="tg"` и `channel="vk"`. Метод `any_enabled` возвращает `True`, если хотя бы один из 8 флагов включён — именно такие пользователи попадают в выборку для фонового мониторинга.

6. Технические решения

6.1. Получение данных

Все данные получаются через API системы «Руобр» (ruobr.ru) с использованием библиотеки ruobr_api (AsyncRuobr). Библиотека обеспечивает асинхронную авторизацию и запросы к эндпоинтам: get_children, get_food_info, get_timetable, get_classmates, get_achievements, get_certificate, get_guide. Приложение реализует обёртку RuobrClient с повторными попытками (max_retries=3, экспоненциальная задержка), таймаутами (30 сек) и обработкой специфических исключений (AuthenticationException, NoChildrenException, NoSuccessException). Для обхода защиты QRATOR (блокирует не-браузерные User-Agent) в main.py выполняется патч httpx._client.USER_AGENT на значение Chrome/120.0.0.0 Safari/537.36.

6.2. Хранение данных

База данных SQLite располагается по пути data/ruobr_bot.db (или задан через DATABASE_URL). Используется пул соединений (5 соединений) с режимом WAL (Write-Ahead Logging) и busy_timeout=5000 мс. Схема включает автоматическую миграцию: при обнаружении старой схемы (chat_id как PRIMARY KEY без колонки id) выполняется полный перенос данных в новую схему с AUTOINCREMENT, включая создание notification_settings из существующих полей users.enabled/marks_enabled/food_enabled. Реализован «healing step» — автоматическое исправление peer_id для VK-only пользователей, у которых chat_id=id после миграции.

6.3. Безопасность

Пароли хранятся исключительно в зашифрованном виде (Fernet, AES-128-CBC). Ключ шифрования (ENCRYPTION_KEY) задаётся через переменную окружения. Все хендлеры и сервисы получают расшифрованные данные через единую точку `safe_decrypt()`, которая затирает пароль из памяти при выходе. Сообщения с паролями в Telegram автоматически удаляются. API-запросы к «Рубр» выполняются через HTTPS. SOCKS5-прокси (при использовании) не имеет аутентификации и ограничивается по IP на уровне фаервола.

6.4. Механизм уведомлений

NotificationService работает как бесконечный цикл с базовым интервалом `CHECK_INTERVAL_SECONDS` (default 300 сек). Пользователи группируются по (login, password_encrypted) для минимизации API-вызовов. Для каждого пользователя применяется случайный jitter (0-300 сек) для равномерного распределения нагрузки.

Тип	Интервал	Условия	Дополнительно
Баланс питания	Совпадает с интервалом питания	11:00-19:00 — каждые 2ч, 20:00-23:00 — каждые 4ч	Порог настраиваемый (default 300₽)
Оценки	15 мин (время уроков), 60 мин (вне уроков)	Автоопределение по расписанию через DailySchedule	Baseline при первом запуске
Питание	2ч (11:00-19:00), 4ч (20:00-23:00)	Конфигурируемо через FOOD_CHECK_INTERVAL_FAST/SLOW	Определяется по state=30 или ordered+
Дни рождения	1 раз в час (в	Mode: «завтра»	Настраиваемый

	заданное время)	(ежедневно) или «weekly» (выбранный день)	день и час
--	-----------------	---	------------

Дедупликация реализуется через таблицу `notification_history` с уникальным ключом (`user_id`, `notification_type`, `notification_key`, `channel`) — каждая комбинация отправляется только один раз. При блокировке бота пользователем (TG или VK) все уведомления для этого канала автоматически отключаются.

6.5. Интеграции

Система	Технология	Назначение
Ruobr API	ruobr_api (AsyncRuobr)	Получение данных об учениках: расписание, ДЗ, оценки, питание, одноклассники, ПФДО, справочник школы. HTTP/HTTPS, авторизация через login/password.
Telegram Bot API	aiogram 3.x (AiohttpSession)	Доставка уведомлений и интерфейс пользователя. Long polling с поддержкой SOCKS5-прокси (aiohttp-socks ProxyConnector).
VK API	vkbottle 4.x	Доставка уведомлений и интерфейс пользователя. Long polling через BotLongpoll. Дедупликация по message.id.
SOCKS5 Proxy	serjs/go-socks5-proxy (Docker)	Проксирование TG-трафика при блокировке Telegram провайдером. Контейнер на VPS

		(213.176.117.199:10808).
--	--	--------------------------

6.6. Программное обеспечение

Компонент	Версия	Назначение
Python	3.12-slim	Основной язык разработки
aiogram	>=3.4.0	Telegram Bot фреймворк
vkbot	>=4.3.0	VK Bot фреймворк
aiohttp	>=0.19.0	Асинхронный драйвер SQLite
cryptography	>=41.0.0	Шифрование паролей (Fernet)
ruobr-api	>=1.0.0	Асинхронный клиент API «Руобр»
aiohttp + aiohttp-socks	>=3.9.0 / >=0.8.0	HTTP-клиент с SOCKS5-прокси
httpx	>=0.24.0	HTTP-клиент для Ruobr API (с патчем UA)
python-dotenv	>=1.0.0	Загрузка переменных окружения из .env
Docker	24+	Контейнеризация и развёртывание

7. Инфраструктурные решения

7.1. Топология развертывания

Система развёрнута на двух серверах с разделением ролей:

Сервер	Роль	Описание
Synology NAS	Основной сервер	Docker-контейнер ruobr-bot (Python 3.12). SQLite-БД (data/ruobr_bot.db). Volume: ./data-prod:/app/data. Сеть: host mode. Часовой пояс: Asia/Novosibirsk (GMT+7).
VPS (213.176.117.199)	SOCKS5-прокси	Docker-контейнер go-socks5-proxy (serjs/go-socks5-proxy). Порт 10808/TCP. Без аутентификации, ограничен по IP.

7.2. Конфигурация Docker

Контейнер бота разворачивается через docker-compose.yml с параметрами: restart=unless-stopped, network_mode=host, env_file=.env (или .env.prod). Данные хранятся в volume ./data-prod:/app/data, обеспечивая персистентность БД и логов при пересборке контейнера. Образ собирается на базе python:3.12-slim.

7.3. Переменные окружения

Переменная	Обязательность	Описание
BOT_TOKEN	Обязателен (если нет VK)	Токен Telegram-бота (от @BotFather)

VK_TOKEN	Обязателен (если нет TG)	Токен VK-бота (от vk.com/dev)
VK_GROUP_ID	Обязателен для VK	ID группы VK (positive integer)
ENCRYPTION_KEY	Обязателен	Ключ шифрования Fernet (base64)
BOT_PROXY	Опционально	URL SOCKS5-прокси (socks5://host:port)
ADMIN_IDS	Опционально	Список ID администраторов через запятую
CHECK_INTERVAL_SECONDS	Опционально (300)	Интервал проверки уведомлений
DEFAULT_BALANCE_THRESHOLD	Опционально (300.0)	Порог баланса по умолчанию
FOOD_CHECK_INTERVAL_FAST	Опционально (7200)	Интервал проверки питания (день)
FOOD_CHECK_INTERVAL_SLOW	Опционально (14400)	Интервал проверки питания (вечер)
LOG_LEVEL	Опционально (INFO)	Уровень логирования
CACHE_TTL_SECONDS	Опционально (300)	TTL кэша расписания
CACHE_MAX_SIZE	Опционально (1000)	Макс. размер кэша
RATE_LIMIT_PER_MINUTE	Опционально (30)	Лимит запросов в минуту

7.4. Журналирование

Логи выводятся в stdout (для docker logs) и в файл data/bot.log. Формат: timestamp - module - level - message. Уровни: INFO для основных модулей (main, database, handlers), WARNING для aiogram/vkbottle/httpx, ERROR для критических ошибок. Для каждого

пользователя логируется входящее сообщение (LoggingMiddleware) и время выполнения хендлера.

7.5. Резервное копирование

База данных SQLite хранится в Docker-volume ./data-prod.

Рекомендуется периодическое резервное копирование файла ruobr_bot.db (и WAL-файла ruobr_bot.db-wal) на внешний носитель или в облачное хранилище. При переносе БД между серверами необходимо выполнить PRAGMA wal_checkpoint(TRUNCATE) перед копированием для консолидации WAL в основной файл.

7.6. Скачивание вложений домашних заданий

Реализована поддержка скачивания и отправки файлов, прикреплённых к домашним заданиям. Бот извлекает файлы из HTML-поля text (regex для <a href> и) и из новых полей API: doc/doc_str (вложения на уровне задачи) и docs_for_lesson (вложения на уровне урока). Файлы скачиваются через авторизованную сессию Ruobr с поддержкой двух режимов: без авторизации (для публичных CDN) и с авторизацией через заголовки username/password. Отправка в Telegram осуществляется через BufferedInputFile (aiogram 3.x): изображения — как photo, документы — как document. Максимальный размер файла — 10 МБ. При невозможности отправки файла предоставляется ссылка на файл в виде текстового сообщения.